



Técnica em Graph Mining

UNIDADE 02

Explorando grafos com programação

| Explorando grafos com programação

Estrutura de dados para grafos

Até agora, representamos a estrutura de um grafo de maneira matemática e gráfica. Por exemplo, considere o grafo G_1 graficamente representado na Figura 1. A partir de uma análise visual, podemos concluir que se trata de um dígrafo, composto por cinco vértices e sete arestas não dirigidas e não ponderadas. De maneira matemática, podemos representar o grafo da Figura 1 utilizando dois conjuntos de elementos: $G_1 = (V, E)$, onde V é o conjunto de vértices, de modo que $V = \{1, 2, 3, 4, 5\}$ e E é o conjunto de arestas composto pelos seguintes pares, $E = \{(1,2), (2,4), (3,1), (3,2), (4,3), (5,1), (5,3)\}$.

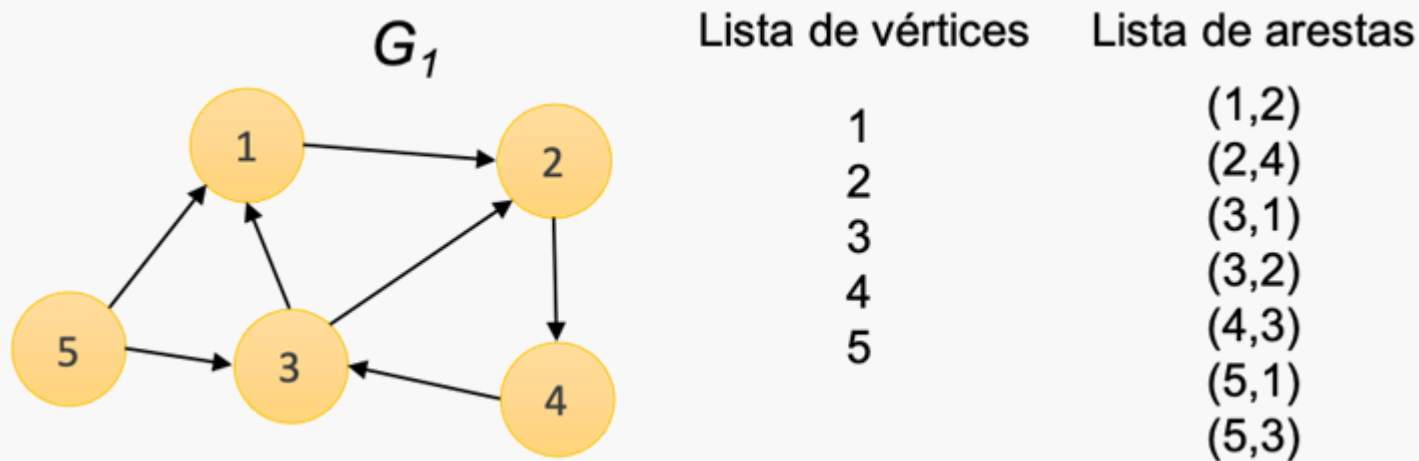


Figura 1. Representação gráfica de um grafo G_1 .

Entretanto, para o uso de grafos em tarefas como a descoberta de padrões inesperados ou a detecção de comunidades utilizando algoritmos de mineração de dados, entre tantas outras tarefas, precisamos representar esta estrutura matemática de modo que os computadores sejam capazes de processá-la. Para isso, podemos utilizar diferentes estruturas de dados que nos permitem representar grafos computacionalmente

com os seus vértices, arestas e as propriedades das arestas, como direção e peso. A seguir, são discutidas as seguintes estruturas de dados: lista de vértices e lista de arestas, lista de adjacência, matriz de adjacência e matriz de incidência. A apresentação destas estruturas é baseada nos materiais de Goldbarg e Goldbarg (2013), Netto e Jurkiewicz (2017) e Simões-Pereira (2013).

Cada uma dessas representações possui vantagens e desvantagens em relação à diferentes critérios e a sua escolha depende do quão denso ou esparso será o grafo que representa o seu problema. Um grafo é dito denso quando o número de arestas se aproxima do número máximo de arestas possível. Por exemplo, um grafo direcionado com N vértices poderá ter no máximo

$$N * (N - 1)$$

arestas. Já um grafo não direcionado terá no máximo

$$N * (N - 1) / 2$$

$$N * N - 1$$

$$N * \frac{N - 1}{2}$$

arestas. Por outro, um grafo é dito esparso quando possui um pequeno número de arestas em relação ao número de vértices. Em geral, os critérios utilizados para a escolha da estrutura adequada levam em consideração a quantidade de memória (ou complexidade de espaço) para armazenar a estrutura do grafo, bem como o tempo para a realização de atividades básicas como verificar se uma determinada aresta faz parte do grafo e localizar os vértices vizinhos de um determinado vértice.

Lista de vértices e lista de arestas

Uma das maneiras mais simples e com menor esforço para representar um grafo é utilizar duas listas, uma para os vértices e outra para armazenar as arestas, similar à representação matemática. No exemplo anteriormente discutido na Figura 1, a lista de vértices é composta pelos elementos do conjunto V, enquanto a lista de arestas é composta pelos elementos do conjunto E. No decorrer desta unidade, a notação |V| será utilizada para denotar a quantidade de elementos no conjunto de vértices V e |E| a quantidade de elementos no conjunto de arestas E.

Com a linguagem de programação Python, uma das maneiras de representar as listas de vértices e arestas é utilizar a estrutura *list* da seguinte maneira:

```
>>> V = [1, 2, 3, 4, 5]
```

```
>>> E = [
```

```
(1,2),
```

```
(2,4),
```

```
(3,1),
```

```
(3,2),
```

```
(4,3),
```

```
(5,1),
```

```
(5,3) ]
```

Caso seja necessário representar um grafo ponderado, podemos adicionar um terceiro elemento nas tuplas do interior da lista de arestas E fornecendo os pesos ou custos das arestas. Veja no exemplo abaixo, as arestas do conjunto E atualizadas com os pesos 0.3, 0.8, 1, 0.6, 1, 0.4 e 0.7, respectivamente.

```
>>> E = [
```

```
(1,2,0.3),
```

```
(2,4,0.8),
```

```
(3,1,1),
```

```
(3,2,0.6),
```

```
(4,3,1),
```

```
(5,1,0.4),
```

```
(5,3,0.7) ]
```

É importante observar que os exemplos acima são de um grafo direcionado. Assim, se existisse uma aresta com origem no vértice 2 e destino no vértice 1, seria necessário a inclusão da tupla (2,1) na lista E, mesmo com a existência do par (1,2). Caso fosse um grafo não direcionado, somente o par (1,2) seria suficiente para representar ambas as direções da aresta envolvendo os vértices 1 e 2.

O consumo de espaço para armazenar a lista de vértices será proporcional a quantidade de vértices do grafo. Em outras palavras, a complexidade de espaço será $O(|V|)$. De maneira similar, a complexidade de espaço da lista de arestas será $O(|E|)$, já que precisamos armazenar cada uma das arestas. Assim, a complexidade de espaço para armazenar ambas as listas desta estrutura de dados será $O(|V| + |E|)$.

A principal desvantagem desta representação é em relação aos custos de tempo associados para a realização de operações essenciais. Por exemplo, para encontrar todos os vértices adjacentes de um dado vértice, ou em outras palavras, encontrar todos os vizinhos de um dado vértice que estão diretamente ligados a ele por uma aresta, é necessário realizar uma busca sequencial (também chamada de busca linear) por toda a lista de arestas verificando para cada par da lista, se o vértice de origem ou de destino, corresponde ao vértice da consulta. Desse modo, o tempo computacional desta tarefa será $O(|E|)$. Outra operação frequente é verificar se dois vértices estão conectados. Novamente, será necessário realizar uma busca sequencial na lista de arestas, verificando todos os possíveis pares. Grafos pequenos, como o exemplo da Figura 1, pode nos levar a acreditar que percorrer toda lista de arestas não é uma tarefa tão custosa. Entretanto, é importante lembrar que em problemas reais precisamos lidar com um grande número de vértices e possivelmente um número ainda maior de arestas.

Para entender melhor a dimensão do problema, considere N como a quantidade de vértices de um grafo. Em um grafo não direcionado, a quantidade máxima de vértices M será de

$$M = N * N - 12$$

$$M = N * \frac{N - 1}{2}$$

. A Figura 2 ilustra essa relação entre a quantidade de vértices e a quantidade máxima de arestas para grafos variando de 2 a 5 vértices. Assim, considerando uma análise de pior caso onde temos um grafo denso em que a quantidade de arestas será igual a quantidade máxima possível, em um grafo com mil vértices ($N = 1.000$), teríamos uma quantidade $M = 499.500$ arestas. Para $N = 10.000$, teríamos uma quantidade $M = 49.995.000$ arestas e assim por diante. Para grafos direcionados, estes valores são ainda maiores (na verdade, o dobro), pois a relação entre a quantidade de arestas e vértices será de

$$N * (N - 1)$$

ou simplesmente

$$(N^2 - N)$$

$$N * N - 1$$

$$N^2 - N$$

se multiplicarmos os termos da equação. Em ambos os casos, a complexidade de tempo no pior caso será quadrática (N^2), já que podemos ignorar os termos de menor ordem. O que é importante ter em mente é que a quantidade M de arestas de um dado grafo será um valor entre $0 \leq M \leq N^2 - N$

$$0 \leq M \leq N^2 - N$$

, se for um grafo direcionado e

$$0 \leq M \leq (N^2 - N)$$

$$0 \leq M \leq \left(\frac{N^2 - N}{2} \right)$$

, se for um grafo não direcionado. Observando estes valores, fica mais claro o quanto é computacionalmente custoso realizar uma busca sequencial por toda a lista de arestas, já que a quantidade de arestas M pode ser valor extremamente elevado.

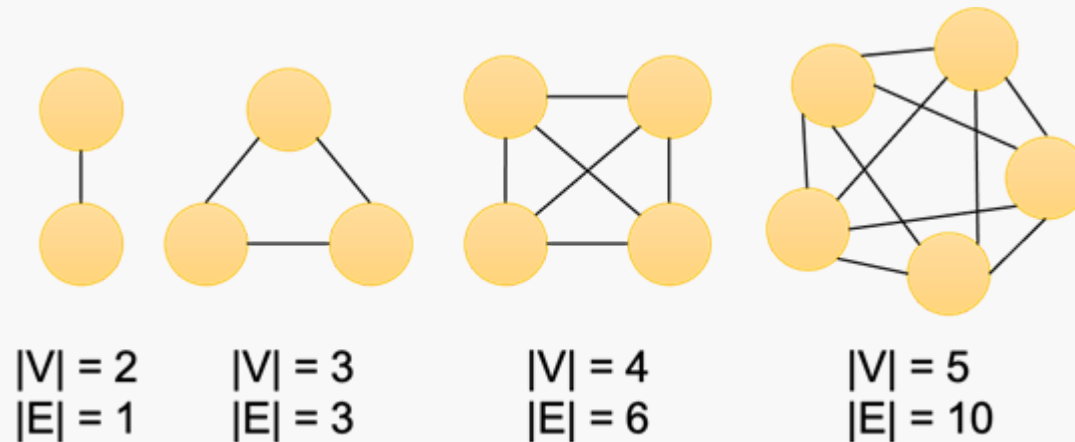


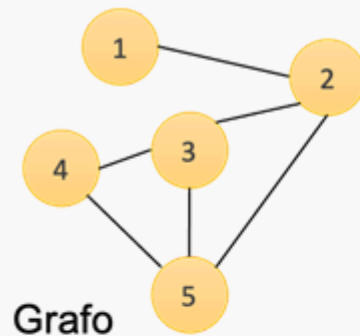
Figura 2. Relação entre o número de vértices e a quantidade máxima de arestas possíveis em grafos não direcionados.

A seguir, apresentamos outras estruturas de dados que são computacionalmente mais eficientes do que as listas de vértices e arestas.

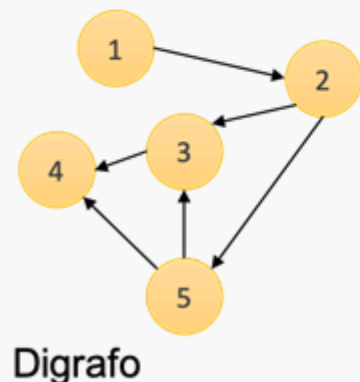
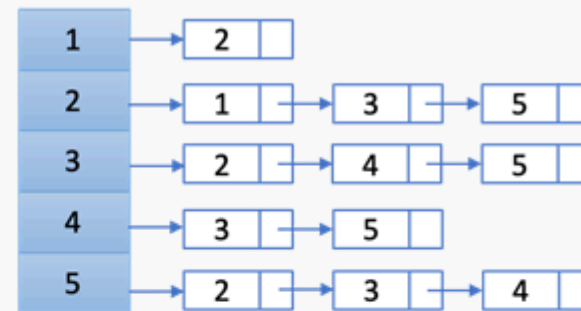
Lista de adjacência

Uma das maneiras mais utilizadas para representar um grafo consiste em indicar para cada vértice, quais os outros vértices estão ligados a ele. Em outras palavras, quais são os vértices adjacentes a ele. Para isso, cria-se uma lista chamada de lista de adjacência com $|V|$ posições, onde cada posição dessa lista possui uma lista encadeada que aponta para as posições dos vértices adjacentes. Se o grafo for direcionado, os vizinhos de um vértice u serão somente aqueles vértices para os quais existe uma aresta saindo de u .

Essa representação utilizando listas encadeadas fica mais clara na Figura 3, onde são ilustrados dois grafos, sendo um não direcionado e outro direcionado (digrafo), ambos com 5 vértices e 6 arestas. Ao lado de cada grafo, é apresentada a lista de adjacência. Podemos observar que do lado esquerdo de cada lista de adjacência, temos uma lista vertical com 5 posições (representadas com um fundo em azul), sendo uma posição para cada vértice do grafo. Para cada posição dessa lista ou para cada vértice, temos uma lista horizontal que aponta para todos os vizinhos deste vértice.



Lista de adjacência



Lista de adjacência

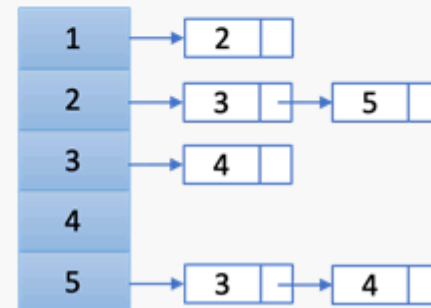


Figura 3. Exemplo de um grafo não direcionado (acima) e direcionado (abaixo) e as listas de adjacência.

Para o digrafo da Figura 3, é possível interpretar que a partir do vértice 1, há uma aresta que o conecta até o vértice 2. A partir do vértice 2, existe uma aresta que o conecta até o vértice 3 e outra aresta que o conecta até o vértice 5. A partir do vértice 3, existe uma aresta que o conecta até o vértice 4. Por fim, a partir do vértice 5, existe uma aresta que o conecta até o vértice 3 e outra aresta que o conecta até o vértice 4. É interessante notar que o vértice 4 não possui nenhuma aresta partindo dele. Por isso, a sua lista encadeada se apresenta como vazia.

Em Python, podemos representar a lista de adjacência utilizando um dicionário de listas, conforme ilustrado abaixo para os dois grafos exemplificados na Figura 3. Esta representação nos diz, por exemplo, que na primeira lista de adjacência existe uma aresta partindo do vértice 1 para o vértice 2, arestas partindo do vértice 2 para os vértices 1, 3 e 5, arestas partindo do vértice 3 para os vértices 2, 4 e 5, e assim por diante.

```
>>> listaAdjacenciaGrafo = {  
1 : [2],  
2 : [1,3,5],  
3 : [2,4,5],  
4 : [3,5],  
5 : [2,3,4] }  
>>> listaAdjacenciaDigrafo = {  
1 : [2],  
2 : [3,5],  
3 : [4],  
4 : [],  
5 : [3,4] }
```

Em grafos não direcionados, são necessárias

$$|V|+(2*|E|)$$

$$V + (2 * E)$$

posições para armazenar a lista de adjacência, já que é necessário armazenar a informação de cada vértice e cada aresta é armazenada duas vezes na lista, uma para cada direção. Por exemplo, para a aresta envolvendo os vértices 1 e 2 do grafo direcionado ilustrado na Figura 3, temos tanto $1 \rightarrow 2$, como $2 \rightarrow 1$ na lista de adjacência, conforme podemos verificar nas duas primeiras linhas da variável *listaAdjacenciaGrafo* apresentado anteriormente. Em grafos direcionados, cada aresta aparece uma única vez na lista de adjacências, apresentando um custo de $|V|+|E|$

$$V + E$$

posições de memória para o seu armazenamento. Assim, em ambos os casos podemos dizer que a complexidade de espaço será $O(|V|+|E|)$.

Embora a complexidade de espaço da lista de adjacência seja similar a da lista de vértices e arestas, os custos de tempo para a realização de operações básicas são reduzidos. Por exemplo, podemos chegar na lista de vizinhos de um determinado vértice em tempo constante $O(1)$, apenas indo até o índice do vértice na lista. Para verificar se dois vértices estão conectados, primeiro precisamos ir até o índice de um dos vértices na lista e, em seguida, buscar pelo segundo vértice na lista encadeada. A primeira operação possui tempo constante $O(1)$, enquanto a segunda operação exige caminhar por uma lista encadeada com no máximo $|V| - 1$ elementos. Assim, o custo total de tempo será $O(|V|)$.

Matriz de adjacência

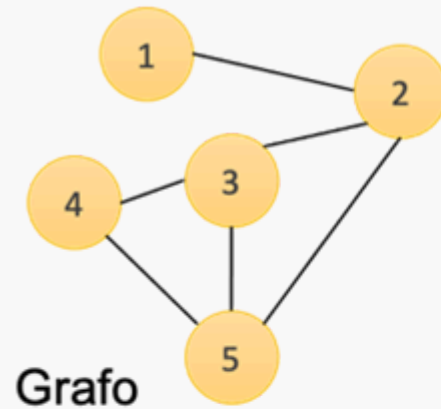
Outro modo conveniente para armazenar a estrutura de um grafo é a partir da matriz de adjacência. Neste caso, utiliza-se uma matriz quadrada de dimensões

$$|V| \times |V|$$

$$V \times V$$

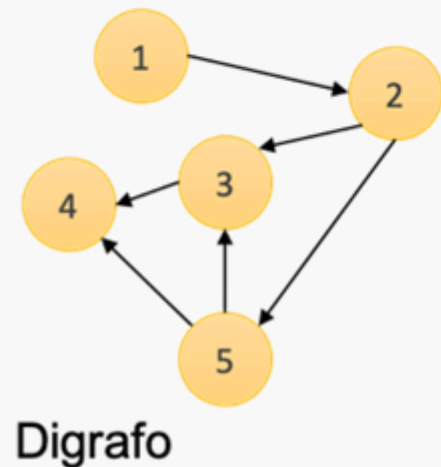
. Cada posição i,j da matriz é preenchida com 1 caso exista uma aresta entre o vértice i e o vértice j , ou preenchida com 0 caso não exista uma aresta entre eles. Caso seja um grafo ponderado, a posição i,j da matriz é preenchida com o peso da aresta que conecta os vértices i e j . Caso não exista uma aresta entre os vértices, pode-se considerar um valor que nunca será associado aos pesos das arestas como $-\infty$ ou ∞ . A Figura 4

ilustra a matriz de adjacência para o grafo direcionado e não direcionado discutidos anteriormente. Note que a matriz do grafo não direcionado é simétrica, mas isso não se aplica para grafos direcionados.



Matriz de adjacência

	1	2	3	4	5
1	0	1	0	0	0
2	1	0	1	0	1
3	0	1	0	1	1
4	0	0	1	0	1
5	0	1	1	1	0



Matriz de adjacência

	1	2	3	4	5
1	0	1	0	0	0
2	0	0	1	0	1
3	0	0	0	1	0
4	0	0	0	0	0
5	0	0	1	1	0

Figura 4. Exemplo de um grafo não direcionado (acima) e direcionado (abaixo) e matrizes de adjacência.

Em Python, podemos representar as matrizes da Figura 4 da seguinte maneira.

```
>>> matrizAdjacenciaGrafo = [
[0,1,0,0,0],
[1,0,1,0,1],
[0,1,0,1,1],
[0,0,1,0,1],
[0,1,1,1,0] ]
>>> matrizAdjacenciaDigrafo = [
[0,1,0,0,0],
[0,0,1,0,1],
[0,0,0,1,0],
[0,0,0,0,0],
[0,0,1,1,0] ]
```

Uma das principais desvantagens da matriz de adjacência é o seu custo de armazenamento mesmo no melhor caso, onde se tem um grafo esparso com poucas arestas. Por se tratar de uma matriz com dimensões

$|V| * |V|$

$V * V$

, essa estrutura possui complexidade de espaço quadrática $O(|V|^2)$. Assim, tanto um grafo denso em que a quantidade de arestas se aproxima do número máximo de arestas possíveis, quanto para um grafo esparso com um pequeno número de arestas em relação ao número de vértices, o custo de espaço será o mesmo em ambos os casos. No caso de grafos esparsos, a maior parte da matriz é preenchida com zeros, o que representa um custo desnecessário. Note que para este caso, a lista de adjacências irá apresentar um menor custo de espaço. Entretanto, para grafos densos, esta estrutura irá consumir uma quantidade menor de memória em relação a lista de adjacência, já que a quantidade de vértices, mesmo que ao quadrado, será menor que a quantidade de arestas.

Em uma matriz de adjacência, o custo computacional para encontrar os vizinhos de um dado vértice é a soma do custo para acessar o vértice na matriz e do custo para percorrer toda a linha correspondente ao vértice buscando pelo valor 1. A primeira operação tem custo constante $O(1)$, enquanto a segunda operação tem custo linear $O(|V|)$, já que são percorridas $|V|$ diferentes posições da matriz. Por simplicidade, podemos

ignorar os custos constantes, totalizando um custo de tempo $O(|V|)$ para realizar a tarefa. Para verificar se dois vértices i, j estão conectados por uma aresta, basta acessar a matriz de adjacência na posição i, j . Assim, o custo computacional desta operação é $O(1)$.

Matriz de incidência

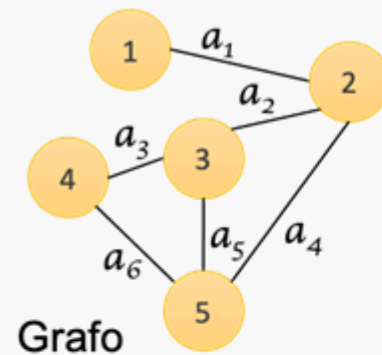
Enquanto a matriz de adjacência mapeia todas as conexões entre pares de vértices, a matriz de incidência relaciona os vértices com as arestas do grafo. Assim, cada linha da matriz corresponde a um vértice e cada coluna corresponde a uma aresta, formando uma matriz de dimensões

$|V| * |E|$

$V * E$

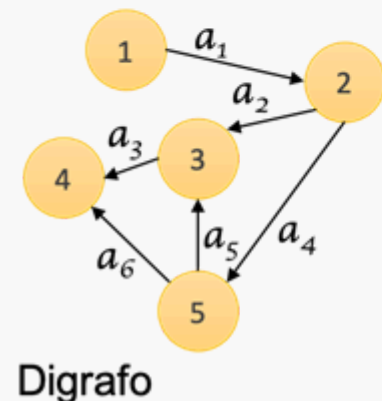
. Nessa matriz, para cada vértice são assinaladas as arestas que estão conectadas ao vértice. Em grafos orientados, considera-se o valor 1 se a aresta tem origem no vértice i , o valor -1 se a aresta tem destino no vértice i ou o valor 0 se a aresta não incide no vértice i . Em grafos não orientados, considera-se apenas o valor 1 quando a aresta incide no vértice i ou o valor 0 quando a aresta não incide no vértice i .

A Figura 5 ilustra dois grafos e suas respectivas matrizes de incidência. Note que nas imagens dos exemplos, as arestas ($a_1, \dots, a_6$) estão indicadas no grafo para facilitar a compreensão das matrizes. No grafo não direcionado, é possível observar que na primeira linha da matriz de incidência correspondente ao vértice 1, somente a aresta a_1 recebe o valor 1, pois é a única aresta incidente a este vértice. Já na segunda linha da matriz, correspondente ao vértice 2, pode-se observar que as arestas a_1, a_2 e a_4 possuem o valor 1, pois todas incidem ao vértice 2. No grafo direcionado, observamos os valores -1, 0 e 1. Por exemplo, na segunda linha da matriz, correspondente ao vértice 2, pode-se observar o valor -1 para a aresta a_1 , pois existe uma aresta com origem no vértice 1 e destino no vértice 2. Também se observa o valor 1 para as arestas a_2 e a_4 , já que estas arestas partem do vértice 2 com destino aos vértices 3 e 5, respectivamente. Em ambos os grafos, denotamos com o valor 0 quando as arestas não incidem um vértice.



Matriz de incidência

	a_1	a_2	a_3	a_4	a_5	a_6
1	1	0	0	0	0	0
2	1	1	0	1	0	0
3	0	1	1	0	1	0
4	0	0	1	0	0	1
5	0	0	0	1	1	1



Matriz de incidência

	a_1	a_2	a_3	a_4	a_5	a_6
1	1	0	0	0	0	0
2	-1	1	0	1	0	0
3	0	-1	1	0	-1	0
4	0	0	-1	0	0	-1
5	0	0	0	-1	1	1

Figura 5. Exemplo de um grafo não direcionado (acima) e direcionado (abaixo) e as matrizes de incidência.

O custo de espaço para armazenar a matriz de incidência é $O(|V| \cdot |E|)$, já que precisamos de $|V|$ linhas e $|E|$ colunas para armazenar a matriz. Para encontrar os vizinhos de um dado vértice, primeiro precisamos percorrer toda a linha do vértice correspondente buscando pelo valor 1, o que representa um custo de $O(|E|)$. Em seguida, para cada aresta encontrada com o valor 1, é preciso percorrer por toda a coluna em busca dos vizinhos, o que representa um custo de $O(|V|)$. Assim, o custo total desta operação será $O(|V| \cdot |E|)$.

Por fim, para verificar na matriz de incidência se dois vértices estão conectados, é preciso percorrer toda a linha de cada um dos vértices do par e verificar se existe uma aresta em comum entre eles, onde ambos possuem o valor 1. Assim, serão realizadas duas buscas com custo $|E|$, totalizando $O(2 \cdot |E|)$. Por simplicidade, podemos ignorar o valor constante e dizer que o custo da operação será $O(|E|)$.

Um resumo sobre as complexidades de espaço e de tempo para a realização das principais operações utilizando as estruturas discutidas nesta unidade, é apresentado na Tabela 1.

Tabela 1. Complexidade de espaço e de tempo ao utilizar uma determinada estrutura de dados para representar o grafo computacionalmente.
Na tabela, $|V|$ representa a quantidade vértices do grafo, enquanto $|E|$ representa a quantidade de arestas.

	Complexidade de espaço	Complexidade de tempo para retornar os vizinhos de um vértice	Complexidade de tempo para verificar se dois vértices estão conectados
Lista de vértices e lista de arestas	$O(V + E)$	$O(E)$	$O(E)$
Lista de adjacências	$O(V + E)$	$O(1)$	$O(V)$
Matriz de adjacência	$O(V ^2)$	$O(V)$	$O(1)$
Matriz de incidência	$O(V * E)$	$O(V * E)$	$O(E)$

| Apresentação da biblioteca NetworkX

Conhecer as principais estruturas de dados para armazenar e processar grafos é fundamental para a escolha de soluções adequadas de acordo com as características e necessidades do problema. Por exemplo, se sabemos que os nossos dados irão gerar um grafo esparso com poucas arestas, utilizar a matriz de adjacência pode não ser a melhor escolha em termos de espaço, já que nesta estrutura há um “desperdício” de memória para armazenar informações referentes a falta de conexão entre vértices. Entretanto, se a quantidade de dados não for tão elevada e

precisamos frequentemente consultar se pares de vértices estão conectados ou não, a matriz de adjacência será a melhor opção devido ao seu baixo custo de processamento. Esse tipo de decisão só é possível quando compreendemos como cada estrutura se comporta e como são os nossos dados.

Anteriormente, foi brevemente discutido como representar os grafos utilizando diferentes estruturas de dados com a linguagem de programação Python. Embora sejam simples para a construção de um grafo, o seu uso na prática requer a implementação de diferentes operações básicas como a inclusão e remoção de vértices e arestas, a busca por caminhos entre vértices, a verificação de conexão entre um par de vértices, entre muitas outras operações. Nesse sentido, o uso de bibliotecas que já possuem estas operações implementadas, representa um ganho de tempo substancial para a manipulação e processamento de grafos. Além disso, estas bibliotecas também disponibilizam algoritmos para a realização de determinadas tarefas e permitem visualizar e personalizar os grafos de maneira simplificada.

Nesta unidade, será introduzida uma dessas bibliotecas chamada NetworkX (HAGBERG; SCHULT; SWART; 2008). A NetworkX é uma biblioteca para a criação, manipulação e estudo de grafos e redes complexas utilizando a linguagem de programação Python. A biblioteca possui centenas de funções que facilitam a criação e manipulação de grafos, sendo capaz de processar conjuntos de dados com mais de 10 milhões de vértices e 100 milhões de arestas. Ainda, a biblioteca conta com uma extensa documentação online em seu website oficial¹ e uma grande comunidade de usuários.

Instalação do Sistema

Para instalar a versão 2.5 da biblioteca NetworkX, preliminarmente é necessário ter instalado no sistema o Python 3.6, 3.7 ou 3.8. A instalação do Python pode ser realizada de diferentes maneiras, como a partir do seu site oficial (<https://www.python.org/downloads/>). Por conveniência, sugerimos a instalação da distribuição Anaconda (<https://www.anaconda.com/products/individual>), que também contém outros pacotes que iremos utilizar. Com a distribuição Anaconda instalada no sistema, a instalação da biblioteca NetworkX é realizada de maneira simplificada. Tanto no *shell* do Linux, como no *bash* do Mac OS X ou no *command prompt* do Windows, basta executar o seguinte comando: `conda install networkx`. Também recomendamos instalar o gerenciador de pacotes *pip* e alguns pacotes adicionais do Python que poderão ser utilizados futuramente como o *numpy*, *scipy*, *pandas* e *matplotlib*. Veja a lista completa de comando a seguir.

```
$ conda install networkx  
$ conda install pip  
$ pip install numpy scipy pandas matplotlib
```

Pronto, agora você já tem um ambiente configurado para começar as suas implementações de grafos utilizando a biblioteca NetworkX.

Primeiros passos na implementação de grafos

Você já pensou na quantidade de funções que precisam ser implementadas para realizar operações básicas utilizando grafos? Por exemplo, imagine que o seu grafo está armazenado em uma lista de adjacência e você precisa remover um determinado vértice. Só para isso, você irá precisar fazer algumas buscas em sua estrutura, além da remoção do vértice em si e de possíveis arestas que ele está conectado. Para que não seja necessário implementar todas estas operações, podemos utilizar bibliotecas como a NetworkX. No vídeo abaixo, apresentamos na prática como realizar algumas operações básicas, mas essenciais, utilizando esta biblioteca.

Primeiros passos na implementação de grafos

PUCPR Online



Assista no

Operações básicas utilizando NetworkX

A seguir, será apresentado um passo a passo para a criação de grafos de diferentes tipos (direcionados e não direcionados, ponderados e não ponderados, multigrafos, entre outros) utilizando a biblioteca NetworkX. Também serão abordadas operações básicas, como adicionar e remover vértices e arestas, atribuir pesos às arestas, fazer consultas às estruturas de dados e visualizar os grafos.

Vamos iniciar com a criação de um grafo não direcionado conforme o exemplo ilustrado na Figura 6.

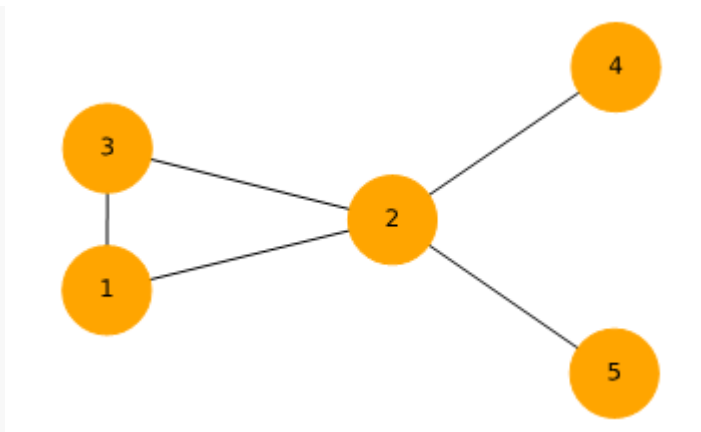


Figura 6. Exemplo de um grafo não direcionado com 5 vértices e 6 arestas.

Os primeiros passos para a criação deste grafo, é a importação da biblioteca e a criação de um grafo vazio utilizando a classe `Graph()`. Iremos atribuir este grafo ao objeto `G` conforme apresentado abaixo.

```
>>> import networkx as nx  
>>> G = nx.Graph()
```

Após a criação do grafo `G`, podemos adicionar vértices e arestas de diferentes maneiras. Por exemplo, podemos adicionar os vértices (chamados de nodes pela biblioteca) de maneira iterativa, um por vez, ou podemos utilizar uma lista. Veja as duas formas ilustradas no exemplo a seguir.

```
>>> G.add_node(1) # adiciona o vértice 1 ao grafo G  
>>> G.add_node(2) # adiciona o vértice 2 ao grafo G
```

```
>>> G.add_nodes_from([3, 4, 5]) #adiciona a lista de vértices [3, 4, 5] ao grafo G
```

No exemplo acima, os vértices são compostos por números inteiros. Mas é importante notar que a biblioteca permite que sejam utilizados diferentes tipos de dados, como um caractere, uma string de caracteres, um arquivo, entre outros.

Para adicionar arestas ao grafo (chamadas de edges pela biblioteca), o procedimento é similar à adição de vértices, também podendo ser realizada de maneira iterativa ou a partir do uso de uma lista de arestas. No caso de lista, é utilizada uma lista de tuplas. Veja o exemplo a seguir.

```
>>> G.add_edge(1,2) # adiciona uma aresta entre os vértices 1 e 2
>>> G.add_edge(1,3) # adiciona uma aresta entre os vértices 1 e 3
>>> G.add_edges_from([(2,3), (2,5), (4,2)]) # adiciona uma lista de arestas considerando os vértices
(2,3), (2,5) e (4,2)
```

Com os códigos acima, o grafo ilustrado na Figura 6 está criado. Para visualizá-lo, basta utilizar a função *draw()* conforme apresentado a seguir. Você irá notar que o código da primeira linha gera um grafo sem apresentar os nomes dos vértices, o que pode dificultar a sua leitura e interpretação. Por isso, na segunda linha apresentamos como apresentar os nomes dos vértices (parâmetro *with_labels*) e fizemos algumas personalizações, como a alteração da cor dos vértices (*node_color*) e do tamanho dos vértices (*node_size*) utilizando a função *draw_networkx()*. Note que a disposição dos vértices do seu grafo será diferente do exemplo ilustrado na Figura 6 e isso irá ocorrer cada vez que você desenhar o grafo. Entretanto, o importante é que as conexões sejam as mesmas.

```
>>> nx.draw(G) # desenho do grafo sem nenhuma personalização
```

```
>>> nx.draw_networkx(G,with_labels=True, node_color='orange', node_size=2000) # desenho do grafo com algumas personalizações
```

No exemplo discutido acima, o grafo foi construído utilizando a classe `Graph()`, que é responsável por construir a estrutura de um grafo não direcionado e ignora arestas múltiplas entre vértices. Por exemplo, se o grafo já possui uma aresta (1,2), a inclusão de uma segunda aresta (1,2) ou (2,1), será ignorada pelo grafo. Para isso, é preciso utilizar outras classes oferecidas pela biblioteca para a criação de grafos direcionados e multigrafos, que permitem laços e arestas paralelas. Além da classe `Graph()`, a biblioteca `NetworkX` possui outras três classes para a criação de grafos de diferentes tipos, sendo elas:

- `DiGraph()`. Essa classe permite a criação de grafos direcionados e permite a realização de operações específicas para esse tipo de grafo;
- `MultiGraph()`. Essa classe permite a criação de grafos não direcionados, assim como a classe `Graph()`, mas permite a inclusão de arestas múltiplas entre dois vértices;
- `MultiDiGraph()`. Essa classe permite a criação de grafos direcionados e arestas múltiplas.

O uso dessas classes é similar ao exemplo apresentado anteriormente. Mas, para que fique mais claro, considere o código abaixo para a criação de um dígrafo `G2`. Note que no grafo `G2`, os vértices são representados por caracteres, *strings* e números. A Figura 7 ilustra o grafo desenhado na última linha de código.

```
>>> G2 = nx.DiGraph() # construção de um grafo direcionado vazio
>>> vertices = ['A','B','C','D','Antonio', 'Matheus', 1] # criação da lista de vértices
>>> arestas = [('A','C'), ('A','D'), ('B','C'), (1, 'A'), ('D','C'),
               ('Matheus','A'), ('Antonio','B'), ('Antonio','C')] # criação da lista de arestas
>>> G2.add_nodes_from(vertices) # adição dos vértices ao grafo G2
>>> G2.add_edges_from(arestas) # adição das arestas ao grafo G2
```

```
>>> nx.draw_networkx (G2, with_labels=True, node_color='orange', node_size=2000) # desenho do grafo G2
```

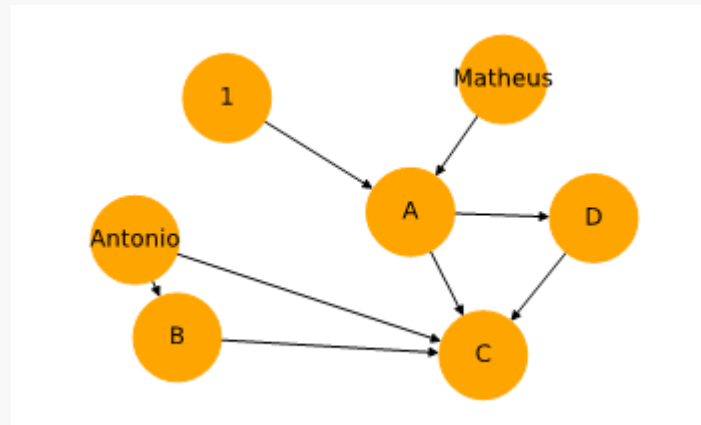


Figura 7. Exemplo de dígrafo construído com a biblioteca NetworkX.

A partir do grafo G2 representado na Figura 7, vamos explorar como realizar algumas operações básicas de exclusão de vértices e arestas. Inicialmente, vamos excluir os vértices *Antonio* e *Matheus* ao mesmo tempo e, então, excluir somente o vértice 1. Para isso, executamos o código apresentado a seguir.

```
>>> G2.remove_nodes_from(['Antonio', 'Matheus']) # removemos uma lista de vértices de G2  
>>> G2.remove_node(1) #removemos somente o vértice 1 de G2
```

É interessante notar que, ao remover um vértice, as arestas associadas a ele também são removidas. Para ter essa confirmação, podemos obter a lista de vértices e a lista de arestas de acordo com o código abaixo.

```
>>> print('Lista de vértices: ', list(G2.nodes))
Lista de vértices:  ['A', 'B', 'C', 'D']

>>> print('Lista de arestas: ', list(G2.edges))
Lista de arestas:  [('A', 'C'), ('A', 'D'), ('B', 'C'), ('D', 'C')]
```

Para remover uma aresta, devemos fornecer como entrada para a função `remove_edge()`, quem são os vértices ligados à aresta. Por exemplo, para remove a aresta ('A', 'C'), executamos o código a seguir.

```
>>> G2.remove_edges('A','C') # remove a aresta composta pelos vértices A e C
```

De maneira similar, se tivéssemos interessados em remover várias arestas de uma única vez, devemos passar a lista de arestas como entrada para a função `remove_edges_from()`. Após este conjunto de operações envolvendo a *remoção de vértices e arestas*, o grafo resultante é ilustrado na Figura 8.

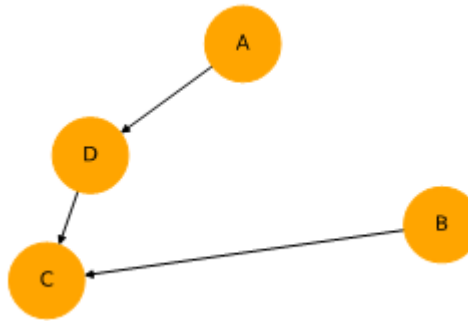


Figura 8. Grafo ilustrado na Figura 7 após a remoção de um conjunto de vértices e arestas.

A partir do grafo ilustrado na Figura 8, vamos obter algumas informações básicas sobre a sua estrutura, como a quantidade de vértices e arestas, calcular o grau dos vértices, calcular o grau de um dado vértice, obter a lista de vizinhos de um determinado vértice, obter a lista de adjacências e a matriz de adjacências. Para isso, considere os códigos abaixo.

```
>>> print('Quantidade de vértices: ', G2.order()) # a ordem do vértice representa a sua quantidade vértices
Quantidade de vértices: 4
>>> print('Quantidade de arestas: ', G2.size()) # o tamanho de um vértice representa a sua quantidade
de arestas
Quantidade de arestas: 3
>>> print('Grau dos vértices: ', G2.degree()) # retorna uma lista com o grau de todos os vértices
Grau dos vértices: [('A', 1), ('B', 1), ('C', 2), ('D', 2)]
>>> print('Grau do vértice C: ', G2.degree('D')) # grau de um vértice específico do grafo
Grau do vértice C: 2
>>> print('Lista de adjacência: \n', G2.adj) # retorna a lista de adjacência
Lista de adjacência:
{'A': {'D': {}}, 'B': {'C': {}}, 'C': {}, 'D': {'C': {}}}
>>> print('Matriz de adjacência: \n', nx.to_numpy_matrix(G2)) # converte o grafo G2 para uma matriz
Matriz de adjacência:
[[0. 0. 0. 1.]
 [0. 0. 1. 0.]
 [0. 0. 0. 0.]
 [0. 0. 1. 0.]]
```

Ainda, podemos realizar algumas consultas como a verificação se um determinado vértice ou aresta pertencem ao grafo, quem são os vizinhos de um determinado vértice, se o grafo é direcionado ou se é um multigrafo. Veja os códigos a seguir.

```
>>> G2.has_node('A') # verifica se o vértice A está na lista de nós
True
>>> G2.has_node('E') # verifica se o vértice E está na lista de nós
False
>>> G2.has_edge('B','C') # verifica se a aresta ('B','C') está na lista de arestas
True
>>> G2.has_edge('C','B') # verifica se a aresta ('C','B') está na lista de arestas
False

>>> G2.is_directed() # verifica se o grafo G2 é direcionado
True
>>> G2.is_multigraph() # verifica se o grafo G2 possui arestas paralelas (multigrafo)
False
>>> print('Vizinhos do vértice A: ', list(G2.neighbors('A'))) # retorna os vizinhos do vértice A
Vizinhos do vértice A: ['C', 'D']
```

Conforme discutimos no decorrer desta unidade, podemos representar os nossos grafos computacionalmente utilizando diferentes estruturas de dados. Assim, também é possível armazená-los em disco de diferentes maneiras, cada uma com vantagens e desvantagens. A seguir, vamos mostrar como armazenar o grafo G2 utilizando a lista de arestas e a lista de adjacência.


```
>>> nx.write_edgelist(G2, 'G2_lista_arestas.txt') # salva o grafo G2 utilizando a lista de arestas
>>> nx.write_adjlist(G2, 'G2_lista_adjacencia.txt') # salva o grafo G2 utilizando a lista de
adjacência
```

A leitura destes arquivos é realizada de maneira similar. A seguir, apresentamos o código para a leitura dos arquivos gerados acima.

```
>>> G2B_arestas = nx.read_edgelist('G2_lista_arestas.txt') # leitura da lista de arestas
>>> G2B_adjacencia = nx.read_adjlist('G2_lista_adjacencia.txt') #leitura da lista de adjacência
```

Todos os exemplos discutidos anteriormente consideram grafos não ponderados. Ou seja, que não possuem pesos associados às arestas. Entretanto, a biblioteca NetworkX nos permite realizar as operações anteriormente discutidas utilizando grafos ponderados. Para isso, precisamos considerar valores de pesos em nossa lista de arestas durante a criação do grafo. O código a seguir ilustra como criar um grafo G3 direcionado e ponderado.

```
>>> G3 = nx.DiGraph()
>>> vertices = ['A','B','C','D','E'] # criação da lista de vértices
>>> arestas = [('A','C',0.2), ('A','B',0.1), ('B','D',0.9),
               ('B','E',0.6), ('D','C',1), ('E','A',0.4)] # criação da lista de arestas
com pesos

>>> G3.add_nodes_from(vertices) # adição dos vértices ao grafo G3
>>> G3.add_weighted_edges_from(arestas) # adição das arestas ponderadas ao grafo G3
```

O código acima constrói um digrafo com 5 vértices e 6 arestas que possuem os respectivos pesos: 0.2, 0.1, 0.9, 0.6, 1 e 0.4. Se utilizarmos a função `draw_networkx()` conforme utilizado anteriormente para desenhar o grafo, não será possível visualizar os pesos das arestas. Para isso, precisamos de alguns códigos adicionais para o posicionamento dos itens na tela e o uso da função `draw_networkx_edge_labels()` conforme ilustrado abaixo.

```
>>> labels = nx.get_edge_attributes(G3,'weight')
>>> pos = nx.spring_layout(G3)
>>> nx.draw_networkx(G3, pos, with_labels=True, node_color='orange', node_size=2000)
>>> nx.draw_networkx_edge_labels(G3, pos, edge_labels=labels)
```

No código acima, a variável `labels` recebe um dicionário com a lista de arestas e seus respectivos pesos. Esse dicionário será posteriormente requisitado pela função `draw_networkx_edge_labels()`. Em seguida, criamos um segundo dicionário na variável `pos`. Neste dicionário são armazenados os nomes dos vértices e em que posição da figura os vértices serão dispostos em termos de coordenadas horizontais e verticais. Para que não seja necessário fornecer um valor de coordenada para cada vértice de maneira manual, podemos utilizar algumas funções que nos retornam estas posições de acordo um layout pré-estabelecido. Uma dessas funções é a `spring_layout()`, que recebe como entrada a estrutura de um grafo e retorna um dicionário com as coordenadas dos vértices. Em seguida, utilizamos a função `draw()` utilizada anteriormente para desenhar o grafo, mas agora com a variável `pos` para indicar a posição dos vértices na imagem. Por fim, utilizamos a função `draw_networkx_edge_labels()` para desenhar as arestas com seus respectivos pesos, sendo necessário informar a posição dos vértices com a variável `pos` e os pesos das arestas com a variável `labels`. Assim, o grafo ponderado `G3` é representado conforma a ilustração na Figura 9.

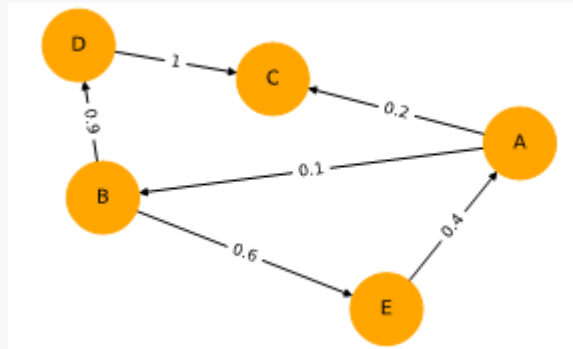


Figura 9. Exemplo de grafo direcionado e ponderado construído com a biblioteca NetworkX.

Conclusão

Esta unidade abordou as principais estruturas de dados para o armazenamento e processamento de grafos. Além disso, também foi introduzida uma biblioteca em Python para a criação e manipulação de grafos chamada NetworkX. A seguir é apresentado um resumo dos principais pontos abordados nesta unidade.

- **Estrutura de dados:** foram apresentadas diferentes estruturas, como lista de vértices e lista de arestas, lista de adjacência, matriz de adjacência e matriz de incidência;
- **Análise das estruturas:** as estruturas foram analisadas levando em consideração o seu custo de armazenamento e o custo para a realização de operações básicas, como encontrar os vizinhos de um determinado vértice ou verificar se dois vértices estão conectados por uma aresta;
- **Introdução ao NetworkX:** foram introduzidas as principais características da biblioteca NetworkX, bem como as configurações necessárias para a sua instalação e uso no sistema;

- **Operações básicas:** foram apresentadas diversas operações frequentemente requisitadas para manipulação de dados utilizando a representação em grafo. Em específico, a biblioteca NetworkX foi utilizada para criação de grafos de diferentes tipos (digrafo e multigrafo), inclusão e remoção de vértices, inclusão e remoção de arestas direcionadas e não direcionadas, com e sem pesos, cálculo de propriedades básicas como ordem e tamanho do grafo, grau de um vértice, consulta da lista de arestas, lista de adjacência, matriz de adjacência, verificação da existência de vértices e de arestas, além da visualização dos grafos.

A partir do estudo desta unidade, você será capaz de selecionar a estrutura de dados adequada ao seu problema, que pode ser composto por grafos esparsos ou densos, de acordo com a quantidade de conexões entre os seus elementos. Além disso, será capaz de realizar os primeiros passos, de maneira independente, na implementação das soluções computacionais envolvendo grafos.

Conclusão

GOLDBARG, M.; GOLDBARG, E. **Grafos: conceitos, algoritmos e aplicações**. Rio de Janeiro: Editora LTC - Grupo GEN, 2012. ISBN 9788595155756. [Minha Biblioteca]. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9788595155756/>

HAGBERG, A. A.; SCHULT, D. A.; SWART, P. J. “**Exploring network structure, dynamics, and function using NetworkX**”, in Proceedings of the 7th Python in Science Conference (SciPy2008), Gäel Varoquaux, Travis Vaught, and Jarrod Millman (Eds), (Pasadena, CA USA), pp. 11–15, Aug 2008.

NETTO, P. O. B; JURKIEWICZ, S. **Grafos: Introdução e prática**. São Paulo: Blucher, 2017. ISBN 9788521211327. [Minha Biblioteca]. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9788521211327/>

SIMÕES-PEREIRA, J. M. S. **Grafos e redes: teoria e algoritmos básicos**. Rio de Janeiro: Editora Interciência, 2013. ISBN 9788571933316. [Biblioteca Virtual Universitária]. Disponível em: <https://plataforma.bvirtual.com.br/Acervo/Publicacao/42049>

